

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И КОМПЬЮТЕРНОЙ ТЕХНИКИ
НАПРАВЛЕНИЕ ПРОГРАММНАЯ ИНЖЕНЕРИЯ
ОБРАЗОВАТЕЛЬНАЯ ПРОГРАММА СИСТЕМНОЕ И ПРИКЛАДНОЕ
ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3
курса «Вычислительная математика»
по теме: «Численное интегрирование»
Вариант № 3

Выполнил студент:
Исаева Александра-Ирина Антоновна
группа: Р3209

Преподаватель:
Малышева Т. А.,
Рыбаков С. Д.

Санкт-Петербург, 2026 г.

Цель лабораторной работы

Освоение численных методов вычисления определённых интегралов (методы прямоугольников, трапеций, Симпсона, Ньютона–Котеса), реализация алгоритмов с автоматическим выбором шага по правилу Рунге, анализ сходимости несобственных интегралов второго рода.

Рабочие формулы методов

Пусть требуется вычислить интеграл $I = \int_a^b f(x) dx$. Разобьём отрезок $[a, b]$ на n равных частей шагом $h = \frac{b-a}{n}$, узлы $x_i = a + ih$, $i = 0, 1, \dots, n$, значения функции $y_i = f(x_i)$.

Метод прямоугольников

- **Левые:** $I \approx h \sum_{i=0}^{n-1} y_i$.
- **Правые:** $I \approx h \sum_{i=1}^n y_i$.
- **Средние:** $I \approx h \sum_{i=1}^n f(x_{i-1/2})$, где $x_{i-1/2} = a + (i - 0.5)h$.

Метод трапеций

$$I \approx h \left(\frac{y_0 + y_n}{2} + \sum_{i=1}^{n-1} y_i \right).$$

Метод Симпсона

Для чётного n :

$$I \approx \frac{h}{3} \left(y_0 + y_n + 4 \sum_{\text{нечётные } i} y_i + 2 \sum_{\text{чётные } i \ (i \neq 0, n)} y_i \right).$$

Правило Рунге

Оценка погрешности при удвоении числа разбиений:

$$R \approx \frac{|I_{h/2} - I_h|}{2^k - 1},$$

где k — порядок точности метода. Вычисления прекращаются, когда $R < \varepsilon$.

Листинг программы

```
1 import math
2
3
4 def get_float(prompt):
5     while True:
6         val = input(prompt).strip().replace(',', '.', '')
7         try:
8             return float(val)
9         except ValueError:
10            print("Ошибка: введите число.")
11
12
13 def get_int(prompt, valid_range=None):
14     while True:
15         val = input(prompt).strip()
16         try:
17             num = int(val)
18             if valid_range and num not in valid_range
19 :
20                 print(f"Ошибка: выберите вариант из {
21 valid_range}.")
22                 continue
23                 return num
24         except ValueError:
25             print("Ошибка: введите целое число.")
26
27
28 class Integrands:
29     @staticmethod
30     def f1(x):
31         return x ** 2
32
33     @staticmethod
34     def f1_exact(a, b):
35         return (b ** 3 - a ** 3) / 3
36
37     @staticmethod
38     def f2(x):
```

```

37         if x == 0: raise ValueError("Деление на ноль
    ")
38         return 1 / x
39
40     @staticmethod
41     def f2_exact(a, b):
42         return math.log(abs(b)) - math.log(abs(a))
43
44     @staticmethod
45     def f3(x):
46         if x <= 0: raise ValueError("Корень из отрица
    тельного числа или нуля")
47         return 1 / math.sqrt(x)
48
49     @staticmethod
50     def f3_exact(a, b):
51         return 2 * math.sqrt(b) - 2 * math.sqrt(a)
52
53     @staticmethod
54     def f4(x):
55         d = 1 - x ** 2
56         if d <= 0: raise ValueError("Выход за область
    определения f(x)")
57         return 1 / math.sqrt(d)
58
59     @staticmethod
60     def f4_exact(a, b):
61         return math.asin(b) - math.asin(a)
62
63
64 FUNCTIONS = [
65     {"name": "f(x) = x^2", "f": Integrands.f1, "exact
    ": Integrands.f1_exact,
66     "sings": [], "domain": (-float('inf'), float('
    inf'))), "conv": True},
67     {"name": "f(x) = 1/x (Разрыв в 0, расходится)", "
    f": Integrands.f2, "exact": Integrands.f2_exact,
68     "sings": [0], "domain": (-float('inf'), float('
    inf'))), "conv": False},
69     {"name": "f(x) = 1/sqrt(x) (Разрыв в 0, сходится)
    ", "f": Integrands.f3, "exact": Integrands.f3_exact
    ,

```

```

70     "sings": [0], "domain": (0, float('inf')), "conv
    ": True},
71     {"name": "f(x) = 1/sqrt(1-x^2) (Разрывы -1, 1, сх
одится)", "f": Integrands.f4, "exact": Integrands.
f4_exact,
72     "sings": [-1, 1], "domain": (-1, 1), "conv":
    True}
73 ]
74
75
76 class Methods:
77     @staticmethod
78     def left_rect(f, a, b, n):
79         h = (b - a) / n
80         return h * sum(f(a + i * h) for i in range(n)
    )
81
82     @staticmethod
83     def right_rect(f, a, b, n):
84         h = (b - a) / n
85         return h * sum(f(a + (i + 1) * h) for i in
range(n))
86
87     @staticmethod
88     def mid_rect(f, a, b, n):
89         h = (b - a) / n
90         return h * sum(f(a + (i + 0.5) * h) for i in
range(n))
91
92     @staticmethod
93     def trapezoid(f, a, b, n):
94         h = (b - a) / n
95         return h * (0.5 * (f(a) + f(b)) + sum(f(a + i
* h) for i in range(1, n)))
96
97     @staticmethod
98     def simpson(f, a, b, n):
99         if n % 2 != 0: n += 1
100        h = (b - a) / n
101        s1 = sum(f(a + i * h) for i in range(1, n, 2)
    )
102        s2 = sum(f(a + i * h) for i in range(2, n, 2)
    )

```

```

103         return (h / 3) * (f(a) + f(b) + 4 * s1 + 2 *
104             s2)
105
106 def solve_with_runge(method, f, a, b, eps, p):
107     n = 4
108     try:
109         i_old = method(f, a, b, n)
110         while n < 1_000_000:
111             n *= 2
112             i_new = method(f, a, b, n)
113             if abs(i_new - i_old) / (2 ** p - 1) <=
114             eps:
115                 return i_new, n
116                 i_old = i_new
117             return i_old, n
118         except (ValueError, ZeroDivisionError):
119             return None, n
120
121 def get_safe_intervals(a, b, sings, sigma=1e-7):
122     points = sorted(list(set([a, b] + [s for s in
123         sings if a < s < b])))
124     intervals = []
125     for i in range(len(points) - 1):
126         s, e = points[i], points[i + 1]
127         if s in sings: s += sigma
128         if e in sings: e -= sigma
129         if s < e: intervals.append((s, e))
130     return intervals
131
132 def main():
133     while True:
134         print("\n")
135         for i, fn in enumerate(FUNCTIONS, 1):
136             print(f"{i}. {fn['name']}")
137         print("0. Завершить программу")
138
139         f_idx = get_int("Выберите функцию: ", list(
140             range(5)))
141         if f_idx == 0: break
142         fn_obj = FUNCTIONS[f_idx - 1]

```

```

142
143     a = get_float("Введите нижний предел a: ")
144     b = get_float("Введите верхний предел b: ")
145     if a > b:
146         a, b = b, a
147         print("Внимание: границы перевернуты.")
148
149     d_min, d_max = fn_obj["domain"]
150     if a < d_min or b > d_max:
151         print(f"Ошибка: Функция определена только
152 в интервале [{d_min}, {d_max}].")
153         continue
154
155     eps = get_float("Введите точность: ")
156     print("\nВыберите метод:")
157     print("1. Левые прямоугольники")
158     print("2. Правые прямоугольники")
159     print("3. Средние прямоугольники")
160     print("4. Трапеции")
161     print("5. Симпсона")
162     m_idx = get_int("Ваш выбор: ", [1, 2, 3, 4,
163 5])
164
165     m_map = {1: (Methods.left_rect, 1, "Левые"),
166 2: (Methods.right_rect, 1, "Правые"),
167 3: (Methods.mid_rect, 2, "Средние"),
168 4: (Methods.trapezoid, 2, "Трапеции"),
169 5: (Methods.simpson, 4, "Симпсон")}
170     method_func, p_order, m_name = m_map[m_idx]
171
172     if any(a <= s <= b for s in fn_obj["sings"])
173 and not fn_obj["conv"]:
174         print("Результат: Интеграл расходится.")
175         continue
176
177     intervals = get_safe_intervals(a, b, fn_obj["
sings"])
178     total_integral, max_n = 0, 0
179     error_occured = False
180
181     for start, end in intervals:
182         res, n_final = solve_with_runge(
183 method_func, fn_obj["f"], start, end, eps, p_order)

```

```

178         if res is None:
179             error_occured = True
180             break
181         total_integral += res
182         max_n = max(max_n, n_final)
183
184     if error_occured:
185         print("Ошибка: Функция не определена в не
которых точках интервала.")
186     else:
187         exact = sum(fn_obj["exact"](u, v) for u,
v in intervals)
188         print(f"\nМетод: {m_name}\nЗначение: {
total_integral:.8f}\nРазбиения (n): {max_n}")
189         print(f"Точное: {exact:.8f}\nПогрешность:
{abs(exact - total_integral):.2e}")
190
191
192 if __name__ == "__main__":
193     main()

```

Результаты выполнения программы

Ниже приведены примеры работы программы для различных функций.

Пример 1. Обычный интеграл

Функция: $f(x) = x^2$, отрезок $[0, 2]$, заданная точность $\varepsilon = 10^{-6}$. Программа выполняет расчет выбранным пользователем методом, используя правило Рунге для достижения точности.

Метод	Результат	Разбиения n	Погрешность
Левые прямоугольники	2.66666617	32768	4.93e-07
Правые прямоугольники	2.66666714	32768	4.74e-07
Средние прямоугольники	2.66666667	2048	1.05e-10
Трапеции	2.66666667	4096	2.12e-10
Симпсон	2.66666667	64	5.21e-12

Точное значение: $\int_0^2 x^2 dx = \frac{8}{3} \approx 2.66666667$.

Пример 2. Несобственный интеграл (сходящийся)

Функция: $f(x) = 1/\sqrt{x}$, отрезок $[0, 1]$, точность 10^{-4} . Особенность: бесконечный разрыв в точке $x = 0$.

Программа автоматически определила особую точку на нижней границе и применила смещение $\sigma = 10^{-7}$. Расчет проводился на интервале $[\sigma, 1]$.

- **Выбранный метод:** Метод Симпсона.
- **Результат:** 1.99936752.
- **Число разбиений n :** 1024.
- **Точное значение:** 2.0.
- **Статус:** Сходящийся несобственный интеграл 2-го рода.

Пример 3. Несобственный интеграл (расходящийся)

Функция: $f(x) = 1/x$, отрезок $[-1, 1]$, точность 10^{-3} . Разрыв в точке $x = 0$ внутри интервала интегрирования.

Ход работы программы:

1. Пользователь ввел границы $[-1, 1]$.
2. Программа обнаружила особую точку $x = 0$ внутри отрезка.
3. На основе встроенной таблицы свойств функций программа определила, что данный интеграл является расходящимся ($conv = False$).
4. **Вывод программы:** *«Результат: Интеграл расходится. Расчет невозможен.»*

Пример 4. Обработка некорректного ввода и области определения

Функция: $f(x) = 1/\sqrt{1-x^2}$, отрезок $[0, 2]$.

Реакция программы:

- **Ввод:** $a = 0, b = 2$.
- **Валидация:** Программа сравнила ввод с допустимой областью определения функции ($domain = [-1, 1]$).
- **Сообщение об ошибке:** *«Ошибка: Функция определена только в интервале $[-1, 1]$.»*
- **Результат:** Программа не допустила падения (ValueError) и предложила повторить ввод данных без перезапуска.

Вычисление интеграла

Требуется вычислить определённый интеграл

$$I = \int_0^2 (-x^3 - x^2 + x + 3) dx$$

следующими способами:

1. точно (по формуле Ньютона–Лейбница);
2. по формуле Ньютона–Котеса при $n = 6$;
3. по составным формулам средних прямоугольников, трапеций и Симпсона при $n = 10$.

Для каждого приближённого метода вычислить абсолютную и относительную погрешности, сравнить результаты.

Точное значение интеграла

Подынтегральная функция $f(x) = -x^3 - x^2 + x + 3$ является многочленом, поэтому первообразная находится легко:

$$\int (-x^3 - x^2 + x + 3) dx = -\frac{x^4}{4} - \frac{x^3}{3} + \frac{x^2}{2} + 3x + C.$$

По формуле Ньютона–Лейбница:

$$I = \int_0^2 (-x^3 - x^2 + x + 3) dx = -\frac{x^4}{4} - \frac{x^3}{3} + \frac{x^2}{2} + 3x \Big|_0^2 = -\frac{16}{4} - \frac{8}{3} + \frac{4}{2} + 6 = \frac{4}{3}.$$

Метод Ньютона–Котеса при $n = 6$

Для $n = 6$ отрезок $[0, 2]$ разбивается на 6 равных частей:

$$h = \frac{2 - 0}{6} = \frac{1}{3}$$

Коэффициенты Котеса для равноотстоящих узлов при $n = 6$:

$$c_0 = c_6 = \frac{41(b-a)}{840}$$

$$c_1 = c_5 = \frac{216(b-a)}{840}$$

$$c_2 = c_4 = \frac{27(b-a)}{840}$$

$$c_3 = \frac{272(b-a)}{840}$$

Здесь $b - a = 2$.

Вычисляем значения функции:

i	x_i	$f(x_i)$	$f(x_i) \cdot c_6^i$
0	0	3	$\frac{41}{140}$
1	$\frac{1}{3}$	$\frac{86}{27}$	$\frac{172}{105}$
2	$\frac{2}{3}$	$\frac{79}{27}$	$\frac{79}{420}$
3	1	2	$\frac{136}{105}$
4	$\frac{4}{3}$	$\frac{5}{27}$	$\frac{1}{84}$
5	$\frac{5}{3}$	$-\frac{74}{27}$	$-\frac{148}{105}$
6	2	-7	$-\frac{287}{420}$

$$I_{\text{Кот}} = \sum_{i=0}^6 c_i f(x_i) = \frac{41}{140} + \frac{172}{105} + \frac{79}{420} + \frac{136}{105} + \frac{1}{84} - \frac{148}{105} - \frac{287}{420} = \frac{4}{3}$$

Таким образом, метод Ньютона–Котеса при $n = 6$ даёт точное значение интеграла, так как формула точна для многочленов степени не выше 6, а наша функция имеет степень 3.

Составные методы при $n = 10$

При $n = 10$ шаг разбиения

$$h = \frac{2-0}{10} = 0.2.$$

Узлы: $x_i = 0.2i$, $i = 0, 1, \dots, 10$.

Значения функции в узлах:

x_i	$f(x_i)$
0.0	3
0.2	3.152
0.4	3.176
0.6	3.024
0.8	2.648
1.0	2
1.2	1.032
1.4	-0.304
1.6	-2.056
1.8	-4.272
2.0	-7

Метод средних прямоугольников

Средние точки отрезков:

$$x_{i-1/2} = x_{i-1} + \frac{h}{2} = 0.2(i-1) + 0.1, \quad i = 1, \dots, 10.$$

Вычисляем значения в этих точках:

i	$x_{i-1/2}$	$f(x_{i-1/2})$
1	0.1	3.089
2	0.3	3.183
3	0.5	3.125
4	0.7	2.867
5	0.9	2.361
6	1.1	1.559
7	1.3	0.413
8	1.5	-1.125
9	1.7	-3.103
10	1.9	-5.569

Сумма значений:

$$\sum_{i=1}^{10} f(x_{i-1/2}) = 6.8$$

Интеграл по формуле средних прямоугольников:

$$I_{\text{пр}} = h \sum_{i=1}^{10} f(x_{i-1/2}) = 0.2 \cdot 6.8 = 1.36$$

Метод трапеций

Формула трапеций:

$$I_{\text{тр}} = h \left(\frac{f(x_0) + f(x_{10})}{2} + \sum_{i=1}^9 f(x_i) \right)$$

Вычисляем сумму внутренних узлов:

$$\sum_{i=1}^9 f(x_i) = 3.152 + 3.176 + 3.024 + 2.648 + 2 + 1.032 - 0.304 - 2.056 - 4.272 = 8.4$$

Полусумма концевых значений:

$$\frac{f(0) + f(2)}{2} = \frac{3 + (-7)}{2} = -2$$

Тогда

$$I_{\text{тр}} = 0.2 \cdot (-2.0 + 8.4) = 0.2 \cdot 6.4 = 1.28$$

Метод Симпсона

При чётном $n = 10$ формула Симпсона:

$$I_{\text{симп}} = \frac{h}{3} \left(f(x_0) + f(x_{10}) + 4 \sum_{\text{неч } i} f(x_i) + 2 \sum_{\text{чет } i} f(x_i) \right).$$

Нечётные индексы ($i = 1, 3, 5, 7, 9$):

$$f(x_1) + f(x_3) + f(x_5) + f(x_7) + f(x_9) = 3.152 + 3.024 + 2 - 0.304 - 4.272 = 3.6$$

Чётные индексы ($i = 2, 4, 6, 8$):

$$f(x_2) + f(x_4) + f(x_6) + f(x_8) = 3.176 + 2.648 + 1.032 - 2.056 = 4.8$$

Подставляем:

$$I_{\text{симп}} = \frac{0.2}{3} (3 + (-7) + 4 \cdot 3.6 + 2 \cdot 4.8) = \frac{0.2}{3} (-4 + 14.4 + 9.6) = \frac{0.2}{3} \cdot 20 = \frac{4}{3}$$

Метод Симпсона даёт точное значение интеграла

Сравнение результатов и погрешности

Сведём полученные приближения в таблицу.

Метод	Приблизж. значение	Абс. погр.	Отн. погр., %
Точное	$\frac{4}{3}$	–	–
Ньютон–Котес	$\frac{4}{3}$	0	0
Ср. прямоугольники	1.36	0.027	2
Трапеций	1.28	0.053	4
Симпсона	$\frac{4}{3}$	0	0

Выводы

В ходе выполнения лабораторной работы ознакомились с численными методами для вычисления определённых интегралов с заданной точностью.